NIGnets: Neural Injective Geometry Networks for Representing non-self-intersecting Geometry

Atharva Aalok
Department of Aeronautics and Astronautics
Stanford University
Stanford, CA 94305
atharva1@stanford.edu

Juan J. Alonso
Department of Aeronautics and Astronautics
Stanford University
Stanford, CA 94305
jjalonso@stanford.edu

Abstract

Existing deep geometry representations do not have the right priors for aerodynamic design optimization, they often fail to mesh and represent excessively large and irrelevant design spaces. This leads to brittle and inefficient design space exploration. We solve this problem by introducing Neural Injective Geometry networks (NIGnets), a new neural architecture that parameterizes injective and closed transformations, allowing us to represent only simple closed curves and surfaces. This enables design optimization procedures working with NIGnets to perform unconstrained optimization on the network weights while maintaining the non-self-intersection property, thereby guaranteeing water-tight meshes. We also compare the representation power and inference speed of NIGnets with normalizing flow models that can potentially offer similar geometric guarantees.

1 Introduction

The field of geometric representation paired with computational design optimization requires designing shapes that minimize or maximize some quantity of interest. Popular applications include aerodynamic and structural design in the aircraft or automotive industry. Traditionally, designs (e.g., airfoils, car body panels, turbine blades) have been represented using analytic methods such as B-splines or NURBS which are intuitive to work. In recent years, deep learning has revolutionized geometry processing through neural representation methods. However, existing representation methods are often inadequate for design optimization as they do not encode the right priors. One major issue is design space blow-up that occurs due to a method being able to represent a much larger design space than is required. A concrete case in point is aerodynamic optimization which is only concerned with the external geometry that interacts with the flow and where the internal structure has no influence on the quantities of interest. Another issue is self-intersection. Most current geometry representations have no fundamental guarantees on representing only non-self-intersecting shapes and may produce physically undesirable shapes during the optimization process. This often breaks downstream tasks in the pipeline such as meshing or prediction of quantities of interest using surrogate models. Therefore, in a large class of shape optimization problems the shapes of interest are simple closed curves or surfaces.

To address these problems, we introduce Neural Injective Geometry networks (NIGnets). NIGnets are a novel neural architecture that provides a mathematical guarantee that the mapping from the geometry parameter space to the physical coordinate space is injective (one-to-one). In the context of a parametric curve or surface, injectivity is equivalent to the non-self-intersection property.

The core innovation of NIGnets lies in the composition of layers derived from Monotonic Neural networks and an injectivity-preserving skip connection architecture. This design ensures that every intermediate transformation step preserves the one-to-one property. By enforcing this topological

Preprint.

constraint directly into the network architecture, we create an expressive geometric parameterization where every point in the network's weight space corresponds to a physically valid, non-self-intersecting shape. This converts a complex, constrained optimization problem into a significantly simpler, unconstrained one over the network weights.

1.1 Contributions

We summarize our main contributions as follows:

- We introduce the Neural Injective Geometry Network (NIGnet) architecture, composed of monotonic layers and injectivity-preserving skip connections, which guarantees a non-self-intersecting mapping for parametric representations.
- We demonstrate the representational power of NIGnets by successfully fitting high-detail 2D curves and 3D surfaces with varying complexity, showing that the guaranteed constraint does not compromise expressivity.
- We compare the training and inference speed of NIGnets with other methods of representing non-self-intersecting geometry that utilize Neural ODEs.

The remainder of this paper is structured as follows: Section 2 reviews related work in neural geometry and monotonic neural networks. Section 3 details the mathematical foundation and architecture of NIGnets. Section 4 briefly discusses the dataset of 2D and 3D geometry used for model training. Section 5 presents the experimental validation on geometric fitting and optimization tasks. Finally, Section 6 concludes the paper.

2 Related Work

Neural geometry representation has attracted a lot of attention in recent years Park et al. (2019); Mescheder et al. (2019); Groueix et al. (2018); Xiao et al. (2020). Several approaches to deep geometry representations have been studied including point clouds Qi et al. (2017a,b), voxel based representations Wu et al. (2015); Riegler et al. (2017) and implicit neural geometry representations using level-sets Park et al. (2019); Mescheder et al. (2019); Chen and Zhang (2019). Work on mesh deformation based methods has also been done Groueix et al. (2018) but these methods essentially patch together different meshes, each individually transformed to represent a part of the final geometry. Recent work on representing highly detailed geometry using Lagrangian approaches has produced great results but these methods use regularization to represent non-self-intersecting shapes, making it a soft constraint Guo et al. (2024). Chen et al. (2025) have developed a new approach to representing geometry by using text prompts and transforming them to a set of valid CAD commands. This approach has also been studied in Yu et al. (2026). But there is no guarantee on the resulting CAD commands being valid and the generated programs may not compile.

Work on representing only non-self-intersecting geometry, especially fitting shapes based on only a boundary point cloud is quite new. One method is to use diffeomorphic flows as done in Yang et al. (2019). In this type of representation we start with a simple initial non-self-intersecting shape that is transformed in time according to an ordinary differential equation to produce a final shape. The Picard-Lindelof theorem guarantees that the resulting geometry produced in this manner will be non-self-intersecting. The time interval of integration is kept fixed in $[0, 1]$ and different shapes correspond to different ODEs that the initial shape is evolved according to. This has been taken advantage of using Neural ODEs in Yang et al. (2019), but there the authors focussed on a latent space representation and not on capturing the fine-grained details of the geometry. Since this approach is based on the concept of the transformation being invertible, the Neural ODE can be replaced with any normalizing flow and produce the desired effect. Though normalizing flows can achieve this outcome, they are architecturally constrained in such a manner that allows efficient inverse and Jacobian computation. For representing non-self-intersecting geometry, the only requirement is the existence of an inverse even though it may be hard to compute. This property can be leveraged to create more general architectures with more representation power.

We work on a similar method of using NeuralODEs to fit single target shapes where our focus and performance criteria will be the ability to fit a single shape in great detail. We focus on all the performance criteria and also perform a thorough comparison with NIGnets.

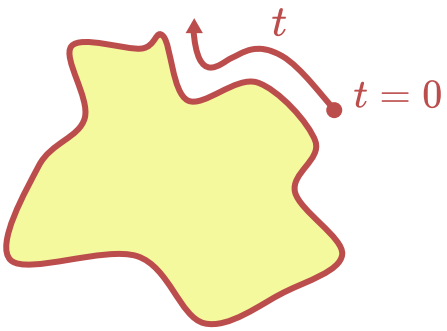
3 Method

We first describe the abstract mathematical idea behind the shape parameterization for representing general simple closed curves in the plane. This approach is conceptual and to generate practical ways of parameterizing such curves a construction method will be needed. Our method describes a specially designed neural network architecture that works as the construction method. We show how the basic multi-layer perceptron (MLP) architecture is a very natural starting point for such architectures.

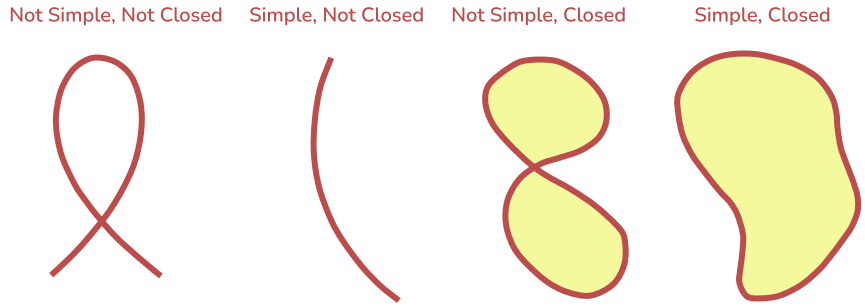
Consider the parameterization in Equation 1 to represent curves:

$$\begin{aligned} x &= f(t) \\ y &= g(t), \quad t \in [0, 1] \end{aligned} \tag{1}$$

As we traverse the parameter t in the interval $[0, 1]$, the functions f and g generate (x, y) coordinate pairs of points on our shape. Figure 1a gives a visual explanation of the parameterization. By sampling t on discrete points in the interval $[0, 1]$ we can generate a point cloud representing our shape.



(a) As we traverse t from 0 to 1, functions f and g trace out a curve in the $x - y$ plane.



(b) We will be concerned with simple closed curves, i.e. curves that loop back and do not self-intersect.

Figure 1: Visualizing the parametric representation and topological constraints of curves.

Given arbitrary f and g, this representation can potentially represent any curve in the plane not necessarily simple and closed, as described in Figure 1b. To constrain the set of curves to only the set of simple closed curves, we require f and g to satisfy the following properties:

Simple A curve is simple if it does not self-intersect. That is, two different t values should not map to the same point in the plane. Mathematically,

$$t_1 \neq t_2 \implies (x(t_1), y(t_1)) \neq (x(t_2), y(t_2))$$

Closed A curve is closed if the starting and the ending points are the same. Mathematically,

$$(x(0), y(0)) = (x(1), y(1))$$

If these additional constraints are satisfied by the functions f and g then the curves generated will be guaranteed to be simple closed curves. We now discuss a way of constructing our functions such that they always satisfy these constraints.

We first take a combined perspective on f and g through a single vector-valued mapping instead of two scalar-valued functions. Consider also, instead of two scalar values x and y, a single vector which has the two values stacked $[x, y]^T$. This vector is parameterized as before through a parameter t. Concretely, consider a vector-valued mapping F that maps t to the vector of x and y as in Equation 2. We now define the conditions that F needs to satisfy to represent only Jordan curves.

$$F : t \rightarrow \begin{bmatrix} x \\ y \end{bmatrix} \tag{2}$$

3.1 Simple Curves

For the curves represented through the mapping F to be simple, we require F to be an injective map. This then maps every unique t value to a different point in the plane and the resulting curve does not self-intersect. Regular MLPs are naturally equipped for this. Consider the specific neural network architecture shown in Figure 2.

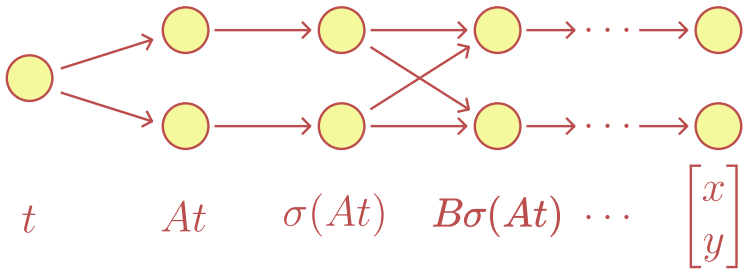


Figure 2: If each layer, that is the linear and nonlinear transformations are separately injective then the composition of these i.e. the entire network will also be injective.

We now consider each transformation of the network. At the first layer the mapping from t to a vector At of $\dim 2$ is performed by a column vector A and is one-to-one as long as $A \neq 0$. The next mapping by a non-linear activation function $\sigma(z)$ is an element-wise mapping. If we choose our activation function to be an injective function, such as the sigmoid, tanh, Leaky ReLU, ELU etc. then the mapping produced from one vector to the next will also be injective. Note here that we cannot use the ReLU as it is not an injective function. Now, consider the mapping from a $\dim 2$ vector to another produced by some 2×2 matrix B. If we generate matrices B at random then, this mapping is injective with probability 1. The neural network is composed of these linear and non-linear mappings in a sequence. Since the composition of two injective mappings is injective, the entire network is injective as a whole. Thus the mapping F from t to $[x, y]^T$ is an injective map and the curves produced by it will be simple curves.

We note that the network hidden layer being limited to a width of 2 is critical as that leads to the matrices B being injective. If we had produced larger hidden layers, say of size m, then we would eventually have to reduced the size back to 2 because our output is a $\dim 2$ vector representing x and y and a $m \times 2$ matrix that would produce this would not be injective. Note that, the network is injective with probability 1, that is if we initialize the network with random weight matrices it will represent a simple curve. This however, does not mean that it is impossible for it to generate self-intersecting shapes, it is just highly unlikely. To make self-intersection impossible refer to 3.3.

3.2 Closed Curves

For the curves represented through F to be closed, we need the x and y values to be the same at $t = 0, 1$. This has to be built into the mapping without affecting the injectivity property that we achieved before. One solution is to first produce the following transformation:

$$C(t) = \begin{bmatrix} cos(2\pi t) \\ sin(2\pi t) \end{bmatrix} \tag{3}$$

This mapping is injective for $t \in [0, 1)$ and also satisfies $C(0) = C(1)$. Therefore, we use this as the first transformation before applying the linear and non-linear activation layers. The modified neural network architecture is shown in Figure 3.

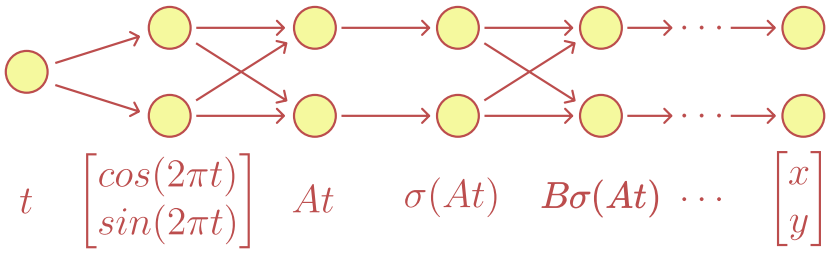


Figure 3: This modified neural network architecture produces closed curves. Also, since the entire network is injective the curves produced will also be non-self-intersecting.

This architecture achieves $F(0) = F(1)$ and the mapping F is also injective with probability 1. We call this component the **Injective Network**.

The point $[\cos(2\pi t), \sin(2\pi t)]$ is a point on the unit circle centered at the origin. The transformation $C(t)$ in Equation 3 maps the open interval $t \in [0, 1]$ to a circle that makes sure that the first and the last point are the same and when fed into the network lead to the same output point and hence create closed curves all while retaining injectivity. This is explained visually in Figure 4.

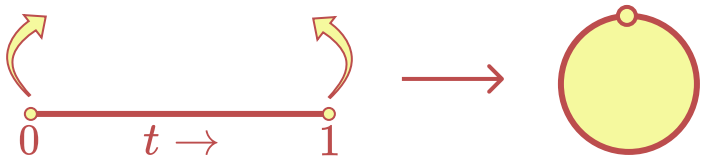


Figure 4: Transforming the line segment $[0, 1]$ to the unit circle centered at the origin injectively.

3.3 Impossible Self-Intersection

We saw in Section 3.1 on simple curves that the network will be injective with probability 1 if the weight matrices are generated at random with some reasonable probability distribution over 2×2 matrices. But this does not mean that self-intersection is impossible, it is just extremely unlikely. It may happen during optimization when we change the network parameters that some of the weight matrices become non-invertible and may lead to self-intersecting shapes.

To guarantee that self-intersection is impossible and not just unlikely we can first take the matrix exponential of the weight matrices in the network and then use them for the linear transformations. The matrix exponential is guaranteed to be invertible for any matrix and hence no matter what our weight matrix is (invertible or non-invertible) the matrix obtained after performing the matrix exponential will be invertible. This gives a hard guarantee on injectivity. Mathematically at every linear layer with input u and output v,

$$W \in \mathbb{R}^{2\times 2}$$
$$A = e^W$$
$$v = Au$$

where, u denotes the current layers activations and v represents the activations after the linear transformation is performed.

This allows us to perform unconstrained optimization over the underlying weights W while retaining A to be injective through the use of the matrix exponential.

3.4 Monotonic Networks

The architecture in Figure 3 can successfully represent only Jordan curves but it is extremely limited in capacity. Each hidden layer only contributes 4 parameters through the A matrix, making the

Injective network parameter deficient as a whole. One way to add parameters to the Network is to use parameterized activation functions e.g. $\sigma(\beta x)$ i.e. the sigmoid with a parameter β or try and create other activations. During the creation of these parameterized activations one needs to keep in mind that injectivity should not be violated for any parameter value to ensure that the network always represents only simple closed curves. It is quite difficult to combine activation functions together with appropriate parameters that add representation power while maintaining injectivity. We will therefore utilize a different more drastic approach where we replace each activation function with an injective neural network.

We start with the realization: a continuous function is injective if and only if it is strictly monotonic. We create injective functions by creating strictly monotonic functions by utilizing neural networks that take in a scalar input and output a scalar value and are strictly monotonic in the input. Mathematically our activation function now is a neural network $M : \mathbb{R} \to \mathbb{R}$ which satisfies one of:

$$\begin{aligned} M(x_1) < M(x_2) \quad \forall x_1, x_2 \in \mathbb{R} \quad \text{s.t.} \quad x_1 < x_2 \\ M(x_1) > M(x_2) \quad \forall x_1, x_2 \in \mathbb{R} \quad \text{s.t.} \quad x_1 > x_2 \end{aligned} \tag{4}$$

We will choose neural networks that by design for any combination of weights are always monotonic, these are called Monotonic Networks. Building Monotonic Networks is an active area of research and there are a few popular choices. The first Monotonic Network was designed by Sill (1997) which used min and max operations to impart monotonicity. Further work on monotonic networks has resulted in new architectures as described in Igel (2023) Daniels and Velikova (2010) and Runje and Shankaranarayana (2023).

In our modified architecuture of Injective networks, every activation layer will be an independent Monotonic Network and all hidden neurons in a layer will use the same network. This architecture is visually described in Figure 5.

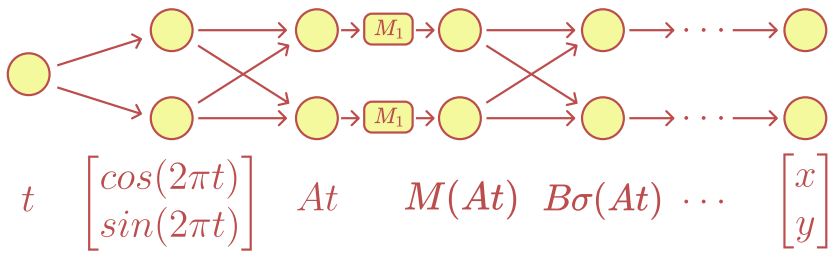


Figure 5: Injective Network with Monotonic Networks as activation functions. M_1 shown in the figure is a monotonic network. The same monotonic network is used for all hidden neurons in a layer.

The $min - max$ operations in the original monotonic architecture by Sill (1997) are not differentiable, are hard to train and also suffer from dead neurons. Recently Igel (2023) proposed a variation of the original Min-Max network proposed by Sill (1997), which replaces the hard $min - max$ with their smooth variants. These are called Smooth Min-Max Monotonic Networks. We will use Smooth Min-Max Monotonic Networks as activation functions to augment Injective Networks. Using the code provided in Igel's repository[1] as reference we develop our own implementation in PyTorch.

The Min-Max and Smooth Min-Max monotonic networks require the weight parameters to be positive, this can be imposed in many ways such as exponential positivity e^z or squared positivity z^2. By default monotonic networks provided in the NIGnets module use squared positivity constraints as they were found to perform slightly better.

3.5 Skip Connections: ResNet Style

The Residual Network (ResNet) architecture as described in He et al. (2016) has made training easier for deeper networks. We can similarly use skip connections with monotonic networks as they preserve monotonicity and hence injectivity while enabling us to train deeper NIGnets. The skip connection connects the input with the output of a subnetwork. We implement this as shown in Figure 6.

We now discuss the validity of skips connections in terms of maintaining injectivity of the layers separately and therefore the network as a whole.

[1] `https://github.com/christian-igel/SMM`

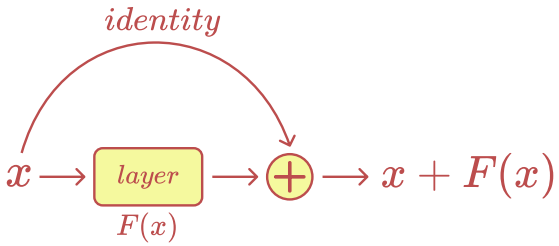


Figure 6: Skip connections connect the input x with the output of a subnetwork $F(x)$. This allows gradients to flow directly from the loss to each layer's weights.

Linear Layers Each linear layer performs the operation $y = Ax$. Adding a skip connection would lead to:

$$y = Ax + x = (A + I)x$$

A skip connection is mathematically unnecessary as we can think of any transformation Ax to be composed of $(B + I)x$ and view B as our weight matrix for the layer instead. In fact using skip connections with the matrix expnential can lead to non-invertible matrices and violate the hard guarantee on injectivity. This is because even though e^A is always invertible, $e^A + I$ is not and may even be the 0 matrix. Thus, we do not use skip connections with the linear layers.

Nonlinear Layers For the nonlinear transformations we use some injective scalar-valued mapping, which is either an activation function or a monotonic network. Assuming that the mapping is an increasing monotonic function $M(x)$, we have the layer's transformation $L(x)$:

$$L(x) = M(x) + x$$

Taking the first derivative we find,

$$L'(x) = M'(x) + 1$$

Since $M(x)$ is an increasing monotonic function, $M'(x)$ is positive and hence we also have,

$$L'(x) >= 1$$

Thus the resulting transformation is also an increasing monotonic function. The derivative being always greater than 1, might be a restriction and we could additionally also include a positively constrained scaling to x as:

$$L(x) = M(x) + \alpha x$$

with,

$$L'(x) = M'(x) + \alpha >= 0$$

In our implementation we use the squared positivity constraint as before that is, the scaling $\alpha^2 x$ with α being the parameter.

In practice, we find that the training dynamics and performance of NIGnets are significantly enhanced by using skip connections and therefore we advise the use skip connections by default with the NIGnets architecture. We elaborate further on the effect of skip connections in the Results section.

3.6 Auxilliary Networks

In the first layer the network transforms the interval $t \in [0, 1]$ to a circle and then deeper layers try to transform that to a desired target shape through constrained 2×2 weight matrices and monotonic networks. The circle may not be the ideal starting shape for the rest of the NIGnet layers to work upon. Further, we can use an unconstrained MLP that replaces the first layer and produces more general simple closed curves to start from. This leads us to the use of Pre-Auxilliary networks.

3.6.1 Pre-Auxilliary Networks

We can replace the first layer transformation to a circle with a more general closed loop represented through polar coordinates in the form:

$$r = f(\theta), \quad \theta \in [0, 2\pi) \tag{5}$$

where $\theta = 2\pi t$ with t in $[0, 1]$.

We can then use the vector $[r\cos(\theta), r\sin(\theta)]^T$ as the first layer and attach the other linear and nonlinear layers on top. This transformation from t to the vector $[r\cos(\theta), r\sin(\theta)]^T$ is injective, satisfies the condition $F(0) = F(1)$ and thus will create simple closed curves. This is powerful as the function f can be represented using a full neural network with no additional constraints except the output being positive. We call this network the **Pre-Auxilliary network**. In practice we observe that the Pre-Auxilliary network provides a favorable initial shape to the Injective Network and making its learning task simpler. From another perspective, Pre-Auxilliary networks can be thought of as a way of augmenting parameters to Injective networks using unconstrained neural networks.

The Pre-Auxilliary network needs to satisfy the following properties:

Positive Output The output of a PreAux Net is the radial coordinate r which is required to be positive. An easy way to ensure this in practice is to use a function like $ReLU$ as the final layer that only produces positive outputs.

Closed Transform The input to the network is θ and for continuity we require: $r(0) = r(\pi)$. This can be ensured in the network design by making the first transformation the mapping from $t \in [0, 1]$ to the unit circle.

Another way to augment parameters to the core Injective network architecture using unconstrained neural networks is through Post-Auxilliary networks which we describe next.

3.6.2 Post-Auxilliary Networks

We can represent the (x, y) output of the Injective network in terms of polar coordinates (r, θ). Using an architecture similar to Pre-Auxilliary networks a first simple idea could be to just add the two polar (r, θ) outputs. One would do this by adding the r value output from the auxilliary network to the different r values that are output from the injective network at any given θ. But performing this directly has a problem. The auxilliary network is defined at all θ by definition but the injective network may not output some values of θ and therefore we cannot add the two. This issue is described visually in Figure 7.

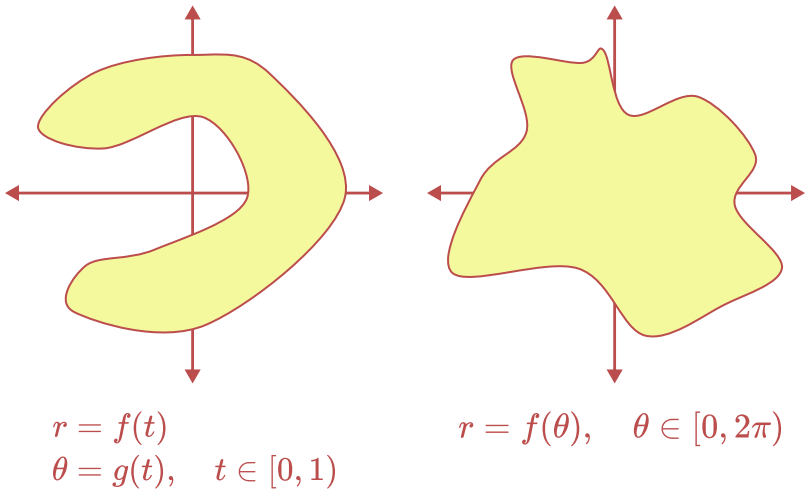


Figure 7: We cannot add the two networks at $\theta = \pi$ since the injective network does not output all possible θ as t varies from $[0, 1)$.

We can bypass this problem by using r values only at t values that lead to valid θ in the Injective network. The valid θ are generated as $g(t)$ from the Injective network. Therefore, the corresponding

$r = h(\theta) = h(g(t))$ are the r values generated from the Post-Auxilliary network at valid θ that occur as t is traversed from $[0, 1)$.

Consider a vector-valued mapping associated with the Post-Auxilliary network:

$$A : t \to \begin{bmatrix} h(g(t)) \\ 0 \end{bmatrix} \tag{6}$$

This requires us to first feed t into the injective network, to get the corresponding θ which can then be fed into the auxilliary network to generate the r for the A mapping. We now have the following two vector-valued mappings for the Injective and Post-Auxilliary networks respectively:

$$F : t \to \begin{bmatrix} f(t) \\ g(t) \end{bmatrix} \quad A : t \to \begin{bmatrix} h(g(t)) \\ 0 \end{bmatrix} \tag{7}$$

We propose to use the addition of these as described before:

$$F + A : t \to \begin{bmatrix} f(t) + h(g(t)) \\ g(t) \end{bmatrix} \tag{8}$$

Following our intuitive construction we now give a proof showing that the mapping generates only simple close curves.

Closed Condition The curves will be closed if $(F+A)(0) = (F+A)(1)$. But we have $F(0) = F(1)$ as F is the injective network. That is,

$$(f(0), g(0)) = (f(1), g(1))$$

We have $A(0) = (h(g(0)), 0)$ and $A(1) = (h(g(1)), 0)$. Now since $g(0) = g(1)$ we also have $A(0) = A(1)$ and hence the curves will be closed.

Simple Curve Condition The curves will be simple if the mapping $F + A$ is injective, which is true if:

$$t_1 \neq t_2 \implies (F + A)(t_1) \neq (F + A)(t_2)$$

We start with the assumption that $F + A$ is not injective. That is for some t_1 and t_2 the outputs are the same. Then we have,

$$g(t_1) = g(t_2)$$

and

$$\begin{aligned} & f(t_1) + h(g(t_1)) = f(t_2) + h(g(t_2)) \\ \implies & f(t_1) + h(g(t_1)) = f(t_2) + h(g(t_1)) \\ \implies & f(t_1) = f(t_2) \end{aligned}$$

But this means that $F(t_1) = F(t_2)$ which is a contradiction as F is the Injective network. Therefore our original assumption is false, and we conclude that the mapping $F + A$ is indeed injective.

Figure 8 shows the architecture for the augmented network.

3.7 Extension to 3D Geometry

The architecture described above can be modified quite simply to generate closed non-self-intersecting surfaces in 3D. The only change required is in the structure of the Injective network. In particular, we need to make the following two changes:

Simple Surfaces: Injectivity The output instead of being a vector $[x, y]^T$ for curves now has to be $[x, y, z]^T$ for surfaces. This means that the network width instead of being constrained to 2 hidden neurons, now has to be 3 hidden neurons.

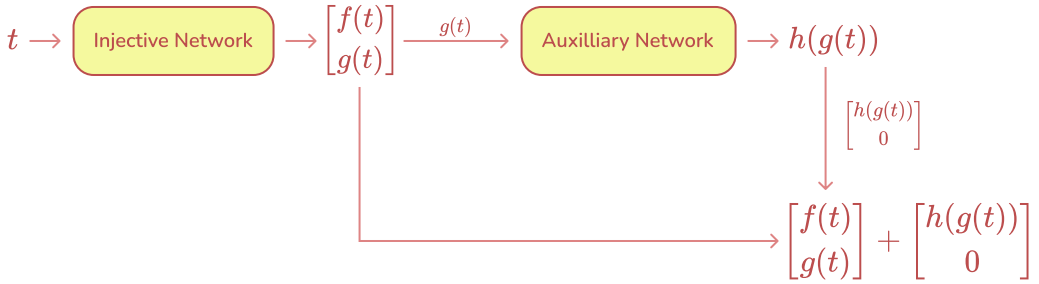


Figure 8: First t is fed into the injective network to obtain its r_i and θ_i output. This θ_i is then fed into the auxilliary network to generate r_a which is then added to r_i to generate the final value.

Closed Condition For tracing out curves in 2D we required a single parameter $t \in [0, 1]$ and the interval $[0, 1]$ was transformed to the unit circle by the initial transformation described in Equation 3. For tracing out surfaces in 3D we need two parameters as a surface in 3D is a 2 dimensional object. Therefore we use parameters $t, s \in [0, 1] \times [0, 1]$. This corresponds to the unit square in the 2D plane. The closed transformation requires us to transform this to the unit sphere as follows:

$$C(t) = \begin{bmatrix} sin(\pi s)cos(2\pi t) \\ sin(\pi s)sin(2\pi t) \\ cos(\pi s) \end{bmatrix} \tag{9}$$

These changes lead to the Injective Network architecture shown in Figure 9.

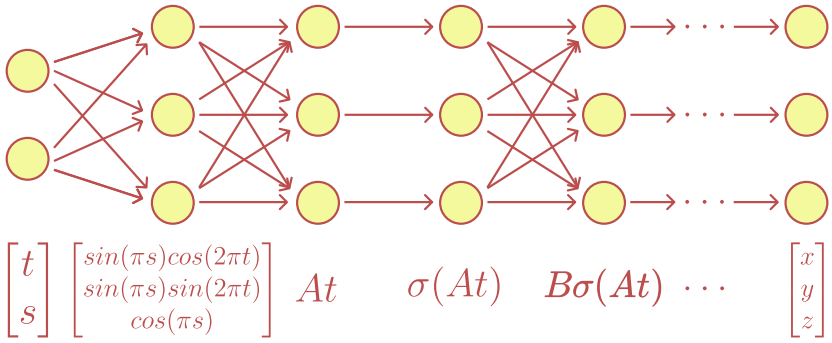


Figure 9: Modified Injective Network architecture that produces simple closed surfaces in 3D. This is an extension of the architecture we developed for producing simple closed curves in 2D.

4 Dataset and Features

We have developed a dataset of target shapes that we would like to test our representation method against. The dataset consists of the following breakdown in terms of shape complexity:

- **Low complexity**: like circle, airfoil, heart etc. to test our implementation and check that things work.
- **Intermediate complexity**: like puzzle piece, aircraft, stanford bunny etc.
- **High complexity**: fractal shapes like Minkowski fractal, star fractal, snowflake fractal etc.

A few sample target shapes are shown in Figure 10.

In particular, the shapes are created as SVG images, which can be sampled at an arbitrary number of points. For more precise fitting of the shapes we can sample more points while trading off training time due to more compute. We make the shapes available online[2].

[2] `https://github.com/atharvaaalok/NIGnets/tree/main/docs/assets/shape_svg`

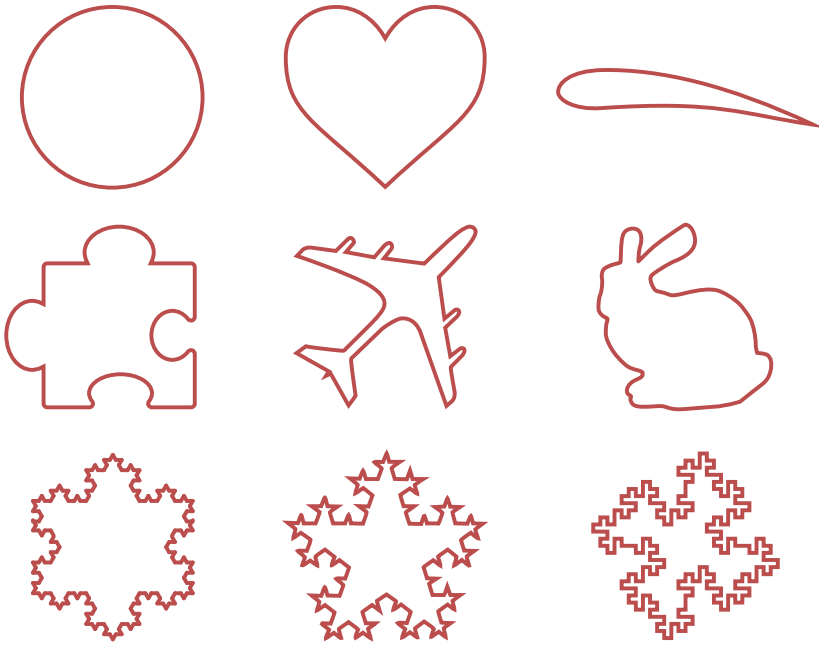


Figure 10: Sample target shapes from the dataset arranged to show low, intermediate and high complexity shapes.

In terms of data-preprocessing we perform centering and uniform scaling in each dimension of the point cloud generated from our images. In particular, after we sample a given number of discrete points from our SVG we move its centroid to the origin and scale the image to lie between the unit hypercube, i.e. $[-1, 1]$ in the longest dimension. We perform the same scaling in each dimension so as to not change the aspect ratio of the shape. We do not perform scaling separately on a per-dimension basis because the goal of our shape networks is to fit the shapes in their undeformed form and be able to handle different length scales in different dimensions.

In this study we compare the performance of NIGnets and Neural ODEs on low and intermediate complexity shapes. Our experiments show that the methods need architectural improvements before they are able to meaningfully tackle the high complexity shapes.

5 Results

We now fit NIGnets to some target shapes to demonstrate their representation capabilities. First, we observe that replacing the usual activation functions with monotonic networks results in a significant improvement in representation power as shown in 11. The same figure also illustrates that representation power is further improved on utilizing skip connections and Pre-Auxilliary networks. Overall we observe a significant improvement in representation capability when utilizing all of: monotonic networks, skip connections and Pre-Auxilliary networks.

We also train NIGnets utilizing all the above mentioned components on the different shapes in the dataset. We find that NIGnet traninig is highly stable and a uniform learning rate of 0.1 and a total of 10,000 training epochs lead to accurate fits of the different target shapes. The results of NIGnet fit to a few different shapes are shown in Figure 12. The networks are composed of 5 layers of Injective network, with smooth min-max monotonic networks with 6-10 groups and 6-10 nodes per group and Pre-Auxilliary networks of 2 layers with 10-25 hidden nodes. We do not utilize Post-Auxilliary networks for these fits.

We also test NIGnets on surfaces in 3D, training on the Stanford bunny and an airplane 3D model. The corresponding fits are shown in 13. In training we observe that fitting models in 3D is significantly harder as compared to 2D as they often get stuck in local optima and require manual and regular hyperparameter tuning to achieve low representation mismatch errors.

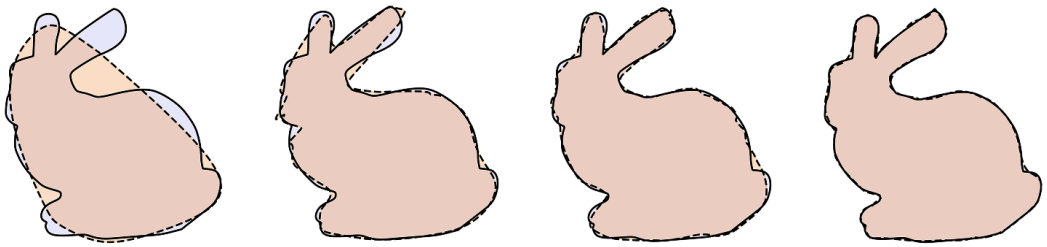


Figure 11: From left to right: 1) Injective network, 2) Injective network with monotonic networks, 3) Injective network with monotonic networks and skip connections, 4) Injective network with monotonic networks, skip connections and Pre-Auxilliary network. We observe a clear improvement in representation accuracy as we add components to the baseline Injective network.

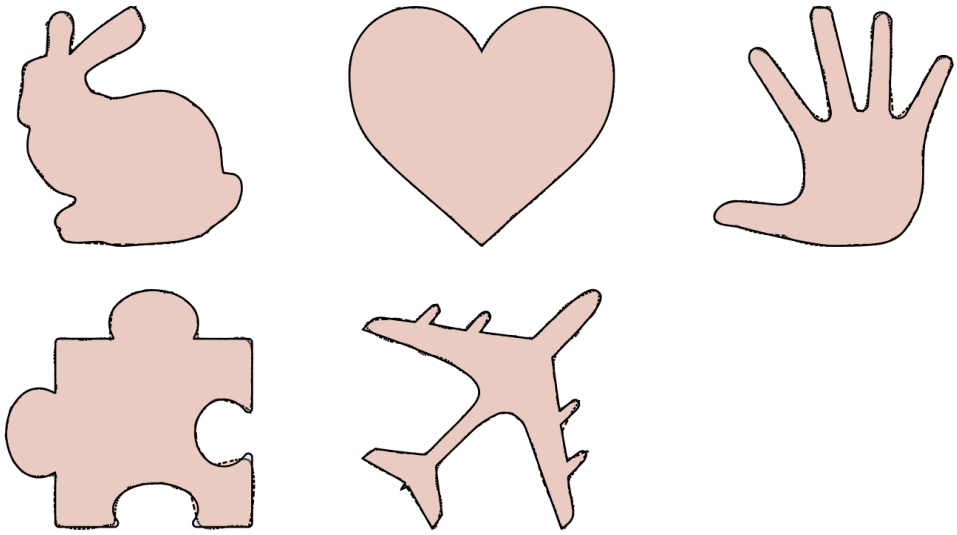


Figure 12: Figures showing NIGnet fit to different shapes in the dataset. Solid line represents target shape and dashed line represents NIGnet fit.

5.1 Training Time

To make more sense of the accuracy of representation we provide below the total training time to fit each target shape once in Table 1.

Shape	NIGnet	Neural ODE
circle	102.5	40.1s
square	99.2s	57.5s
heart	100.9s	52.3s
stanford bunny	105.5s	88.1s
hand	108.3s	125.0s
puzzle piece	105.6s	99.4s
airplane	105.1s	127.2s

Table 1: A comparison of NIGnets and Neural ODEs based on training time.

We observe that NIGnets are extremely consistent in their training times, this is because they are an explicit representation method which directly transforms an input to an output using an analytically defined function. Whereas, Neural ODEs show a high variance in training time. This is because of the adaptive step size used in numerical integration. The integration time depends on the exact function represented by the ODE network. We observe higher training times for more complex shapes, which

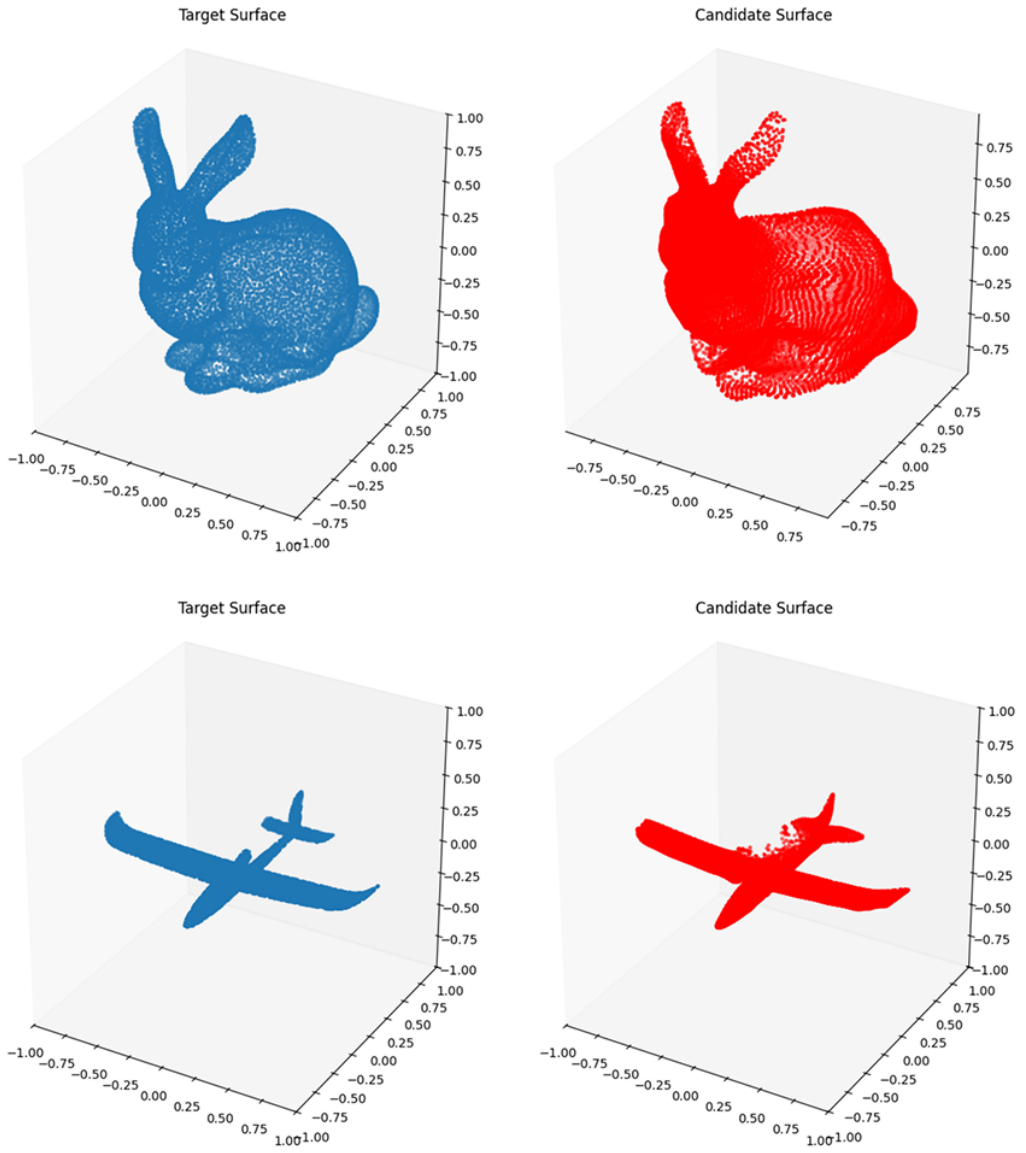


Figure 13: Using NIGnets with 3 hidden layers for representing surfaces surfaces in 3D. Blue point cloud represents the target and the red point cloud is sampled from the NIGnet model fit.

aligns with the intuition that these would require more complex ODE networks that need smaller steps during integration to achieve low integration error.

5.2 Inference Time

Inference time is the time taken to transform an input to the shape represented by our geometry representation method. Since the inference time depends on the discretization resolution (a finer discretization requires more compute and time) we compare how the inference time compares and scales for both NIGnets and Neural ODEs. This comparison is shown in Table 2.

These results were obtained by first fitting the same sized (2800 parameter) NIGnet and Neural ODE models to the stanford bunny as the target shape. Then the model was queried 100 times on different discretization resolutions and the total time reported is the average time over these 100 runs. We observed empirically that at 100 runs the variance of the average time was low enough to provide a reasonable estimate.

Num Points on Shape	NIGnet	Neural ODE
10	0.00085s	0.01552s
100	0.00142s	0.02059s
1000	0.00336s	0.03674s
10000	0.01705s	0.13668s
100000	0.11853s	0.78579s
1000000	1.13743s	10.23218s

Table 2: A comparison of NIGnets and Neural ODEs based on inference time averaged over 100 runs.

We observe that both NIGnets and Neural ODEs scale almost linearly with the discretization resolution though the variation for NIGnets is more regular. We also observe that NIGnets consistently offer a 10x speed boost compared to Neural ODEs making them superior for optimization tasks.

An important consideration for this comparison for Neural ODEs is the integration scheme used. We use an adaptive 5th order Runge-Kutta method. A different integration scheme would lead to a different set of step sizes being used and consequently different inference times and also different final representations. Since NIGnets are an explicit representation defined analytically, they do not suffer from this issue.

6 Conclusion and Future Work

Through this work, we introduce Neural Injective Geometry networks (NIGnets), a neural architecture to represent mathematically guaranteed non-self-intersecting geometry. We achieved this by baking into a baseline MLP architecture the injective and closed properties. We also introduced several concepts that improve the representation capability of these networks such as the use of Pre- and Post-Auxilliary networks. The architecture can also be extended to 3D allowing us to represent simple closed surfaces. We further showed advantages over methods like Neural ODE based geometry representation such as training time consistency and much faster inference speed. Though the representation is quite robust to hyperparameters in 2D we find that it struggles in 3D, often getting stuck in local optima requiring manual tuning of the training process. Lucrative ideas for further research could involve iResNets Behrmann et al. (2019) to perform the discrete deformation version of Neural ODEs. This will allow consistent compute times in training and also speed up inference times. Developing generative models using the NIGnet architecture is also an alternate interesting research challenge.

7 Acknowledgements

Atharva would like to acknowledge the many fruitful discussions with Arjun Shah and Aurelien Ghiglino that formed the basis of several ideas discussed in this work. He also wishes to thank Mr. Manny Ko, Prof. Rafe Mazzeo and Prof. Leonid Ryzhik for their insights on constructing the architecture.

References

Atharva Aalok and Juan J Alonso. 2025. NIGnets. `https://github.com/atharvaaalok/NIGnets`

Jens Behrmann, Will Grathwohl, Ricky TQ Chen, David Duvenaud, and Jörn-Henrik Jacobsen. 2019. Invertible residual networks. In *International conference on machine learning*. PMLR, 573–582.

Cheng Chen, Jiacheng Wei, Tianrun Chen, Chi Zhang, Xiaofeng Yang, Shangzhan Zhang, Bingchen Yang, Chuan-Sheng Foo, Guosheng Lin, Qixing Huang, et al. 2025. Cadcrafter: Generating computer-aided design models from unconstrained images. *arXiv preprint arXiv:2504.04753* (2025).

Zhiqin Chen and Hao Zhang. 2019. Learning implicit fields for generative shape modeling. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 5939–5948.

Hennie Daniels and Marina Velikova. 2010. Monotone and partially monotone neural networks. *IEEE Transactions on Neural Networks* 21, 6 (2010), 906–917.

Thibault Groueix, Matthew Fisher, Vladimir G Kim, Bryan C Russell, and Mathieu Aubry. 2018. A papier-mâché approach to learning 3d surface generation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 216–224.

Minghao Guo, Bohan Wang, Kaiming He, and Wojciech Matusik. 2024. Tetsphere splatting: Representing high-quality geometry with lagrangian volumetric meshes. *arXiv preprint arXiv:2405.20283* (2024).

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.

Christian Igel. 2023. Smooth Min-Max Monotonic Networks. *arXiv preprint arXiv:2306.01147* (2023).

Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. 2019. Occupancy networks: Learning 3d reconstruction in function space. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 4460–4470.

Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. 2019. Deepsdf: Learning continuous signed distance functions for shape representation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 165–174.

Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. 2017a. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 652–660.

Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J Guibas. 2017b. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. *Advances in neural information processing systems* 30 (2017).

Gernot Riegler, Ali Osman Ulusoy, and Andreas Geiger. 2017. Octnet: Learning deep 3d representations at high resolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 3577–3586.

Davor Runje and Sharath M Shankaranarayana. 2023. Constrained monotonic neural networks. In *International Conference on Machine Learning*. PMLR, 29338–29353.

Joseph Sill. 1997. Monotonic networks. *Advances in neural information processing systems* 10 (1997).

Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 2015. 3d shapenets: A deep representation for volumetric shapes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 1912–1920.

Yun-Peng Xiao, Yu-Kun Lai, Fang-Lue Zhang, Chunpeng Li, and Lin Gao. 2020. A survey on deep geometry learning: From a representation perspective. *Computational Visual Media* 6, 2 (2020), 113–133.

Guandao Yang, Xun Huang, Zekun Hao, Ming-Yu Liu, Serge Belongie, and Bharath Hariharan. 2019. Pointflow: 3d point cloud generation with continuous normalizing flows. In *Proceedings of the IEEE/CVF international conference on computer vision*. 4541–4550.

Nomi Yu, Md Ferdous Alam, A John Hart, and Faez Ahmed. 2026. GenCAD-Three-Dimensional: Computer-Aided Design Program Generation Using Multimodal Latent Space Alignment and Synthetic Dataset Balancing. *Journal of Mechanical Design* 148, 3 (2026), 031703.